

Informe de progreso I

Adrián Díaz García

1 CONTEXTO Y CONOCIMIENTOS PREVIOS

L EAGUE of Legends es un videojuego competitivo por equipos en el que cada partida enfrenta a dos equipos de cinco jugadores: el equipo azul y el equipo rojo. El objetivo final es destruir el nexo enemigo, la estructura principal de la base rival. Para alcanzar este objetivo, los equipos deben obtener recursos, controlar zonas del mapa, destruir estructuras defensivas y disputar objetivos neutrales.

En este TFG no se intenta predecir directamente qué equipo ganará la partida, sino tratar de anticipar patrones tempranos de juego a partir de la información disponible antes de que la partida comience.

1.1 La fase de draft

Antes del inicio de la partida, ambos equipos pasan por una fase denominada como draft. En esta fase, los equipos seleccionan los personajes, llamados campeones, con los que jugarán durante la partida, y también bloquean otros campeones para impedir que sean elegidos. Además, el draft también incluye la asignación de cada campeón a una función dentro del equipo.

Esa función se denomina rol. Un rol puede entenderse como la responsabilidad que ocupa un jugador dentro del equipo y la zona del mapa en la que empieza la partida. En League of Legends existen cinco roles: toplaner, jungler, midlaner, botlaner y support.

El draft es especialmente importante para este proyecto porque es información disponible antes del minuto cero. Además, condiciona parcialmente la forma en que cada equipo puede afrontar la partida: algunos campeones favorecen estilos de juego más agresivos, otros composiciones defensivas, otros facilitan el control de los objetivos y otros permiten moverse con mayor facilidad por el mapa. Por esto, el draft será la entrada principal del modelo.

1.2 Mapa, líneas y roles

El mapa de League of Legends es el terreno de juego donde esos roles se despliegan durante la partida. Aunque todos los jugadores pertenecen a un mismo equipo, no empiezan en la misma zona del mapa. Los cinco jugadores se reparten entre tres líneas principales y varias zonas intermedias denominadas jungla.

Las tres líneas principales son la línea superior, la línea central y la línea inferior:

- 1) La línea superior está ocupada por el toplaner;
- 2) la línea central, por el midlaner;
- 3) y la línea inferior, por dos jugadores: el botlaner y el support.

Entre las líneas se encuentran las zonas de jungla, con caminos internos, recursos y accesos hacia distintas partes del mapa. El jungler es el rol que comienza en esta zona y se desplaza por ella para intervenir en las diferentes líneas.

Además de las líneas y la jungla, el mapa también está atravesado por una zona central llamada río. El río conecta distintas regiones del mapa y en sus alrededores aparecen algunos de los objetivos neutrales más importantes.



Fig. 1. Minimapa de League of Legends. Podemos observar ambas bases, las tres líneas y la zona que las separa, conformada por la jungla y el río (la diagonal opuesta a Midlane, en azul).

1.3 Movimiento temprano y roaming

Una partida de League of Legends suele durar aproximadamente entre 25 y 35 minutos, aunque puede terminar antes o alargarse según como se desarrolle. Los primeros minutos tienen una estructura relativamente estable ya que, hasta aproximadamente el minuto 14, la partida se encuentra en la denominada fase de líneas. Durante esta fase, la mayoría de los jugadores permanecen cerca de su zona inicial, acumulan recursos y compiten contra sus rivales directos de rol en la misma línea. Los jugadores de toplane, midlane y botlane tienden a ocupar sus respectivas líneas durante este tramo inicial.

Los dos roles con mayor movilidad al principio son el jungler y el support. El jungler se desplaza por la jungla desde el inicio de la partida, por lo que su movimiento entre zonas forma parte de su comportamiento habitual. El support, en cambio, comienza normalmente acompañando al botlaner en la línea inferior, pero puede abandonar antes esa zona para ayudar en otras partes del mapa.

A este movimiento del support fuera de la línea inferior se le denomina roaming. Un support puede roamear para ayudar en la línea central, acompañar al jungler, obtener información sobre movimientos enemigos en el río o

participar en la disputa de objetivos neutrales. La frecuencia y utilidad de estos movimientos depende, entre otros factores, del campeón elegido, del estado de la partida y del momento temporal analizado.

1.4 Objetivos neutrales

Los objetivos neutrales son recursos importantes que no pertenecen inicialmente a ningún equipo. Para conseguirlos, los jugadores deben desplazarse a zonas concretas del mapa y pelear por ellos.

Entre estos objetivos se encuentran el dragón, situado en la parte inferior del río, y las larvas del vacío, situadas en la parte superior. Estos objetivos son relevantes porque influyen en el movimiento de los jugadores. Por ejemplo, si un support abandona botlane para ayudar a su equipo en el río o en la zona de las larvas, su comportamiento observado se aproxima más a un perfil de roaming.

Por tanto, la posición de los objetivos neutrales ayuda a interpretar los movimientos tempranos de los jugadores y, en particular, los desplazamientos del support.

2 RESUMEN DEL PROYECTO

2.1 Objetivo general

El objetivo inicial del TFG era construir un sistema de aprendizaje automático capaz de predecir patrones de juego observables en partidas de League of Legends a partir de información disponible antes de que comience la partida. La propuesta original planteaba tres patrones principales: comportamiento del jungler, perfil de roaming del support y tendencia espacial del equipo.

La salida final esperada sigue siendo un prototipo por terminal donde un usuario pueda introducir manualmente campeones, roles y variables pre-partida adicionales, y obtener una lectura interpretable de las tendencias esperadas antes del inicio de la partida.

2.2 Relación con trabajos previos

Parte de la literatura sobre League of Legends se ha centrado en predecir la victoria [1] o recomendar campeones durante la fase de draft [2]. En cambio, este TFG no busca estimar directamente el resultado final de la partida, sino aprovechar la señal del draft para aproximar comportamientos observables en el mapa. Esta orientación se alinea con trabajos que estudian patrones de comportamiento, información espacial y evolución temporal en MOBAs [3]-[5].

2.3 Datos utilizados

Para este trabajo se utiliza la API de Riot Games, que proporciona información sobre las partidas que se juegan a diario de League of Legends. En el proyecto se utilizan principalmente dos tipos de datos: la información del draft y las timelines de partida.

La información del draft contiene los campeones seleccionados, los roles asignados y la composición de cada equipo. Esta información es la entrada principal del modelo, ya que está disponible antes de que comience la partida. La timeline, en cambio, describe la evolución de la

partida a lo largo del tiempo. Puede entenderse como una secuencia de estados o “fotografías” de la partida, donde aparecen posiciones, recursos y eventos en cada minuto.

En el enfoque actual, la timeline no se usa como entrada del modelo, sino para construir las etiquetas observadas. Es decir, el modelo predice a partir del draft, pero las respuestas que aprende se calculan observando lo que ocurrió realmente durante los primeros minutos. Esta separación es fundamental: el input del modelo es información pre-partida, mientras que la etiqueta se calcula a posteriori. Por tanto, el objetivo no es predecir decisiones exactas de jugadores concretos, sino estimar hasta qué punto una composición de campeones permite anticipar tendencias tempranas observables de juego.

2.4 Cambio metodológico

Tras la propuesta inicial se construyó un primer pipeline completo de recolección, procesamiento, etiquetado, entrenamiento y validación. Sin embargo, los primeros experimentos mostraron que la dificultad principal no estaba únicamente en la arquitectura del modelo, sino en la definición de las etiquetas.

Concretamente, plantear el problema como una tarea de clasificación generaba una pérdida importante de información. En clasificación, todos los errores penalizan igual: fallar entre dos clases cercanas se trata igual que fallar entre clases opuestas. En cambio, en regresión continua, el error se mide según la distancia entre la predicción y el valor real. Esto encaja mejor con etiquetas que originalmente ya se definen como puntuaciones continuas.

Por este motivo, el enfoque actual reformula el problema como regresión. En esta primera fase se trabaja con un score continuo de roaming del support en el rango [0, 1]. Esta reducción de alcance no elimina las tareas de jungla y equipo del objetivo final, sino que permite validar primero, de forma controlada, la metodología, los datos, la etiqueta y el modelo. Una vez estabilizada la tarea de support, se reintroducirán nuevas versiones continuas de las etiquetas de jungla y equipo.

Así, el avance principal de esta fase es haber convertido una primera propuesta basada en clasificación en un pipeline de regresión, lo que tiene más sentido con la naturaleza continua de las etiquetas.

3 OBJETIVOS INICIALES Y ALINEACIÓN

La propuesta inicial definía como objetivo principal inferir patrones de juego tempranos observables a partir del draft. Ese objetivo se mantiene, pero se ha reformulado la manera de abordarlo. Inicialmente se planteó una clasificación multi-output con tres salidas: comportamiento del jungler, perfil de roaming del support y tendencia espacial del equipo. Actualmente, el trabajo se centra en una primera tarea de regresión continua para la etiqueta de roaming del support.

Esta reducción de alcance no implica un cambio en el objetivo final del TFG, sino una reordenación metodológica. Antes de ampliar el sistema a varias salidas, se busca validar de forma rigurosa una primera etiqueta continua,

comprobar que contiene señal aprendible y construir un pipeline experimental más estable.

3.1 Objetivos que se mantienen

Los siguientes objetivos aún están presentes en el TFG:

- 1) Usar información pre-partida como entrada del modelo.
- 2) Predecir patrones de juego observables, no el resultado final de la partida.
- 3) Elaborar una salida final interpretable para los jugadores.
- 4) Posibilidad de extender el sistema a más etiquetas.
- 5) Crear un prototipo por terminal que permita introducir un draft y obtener una lectura comprensible.

3.2 Objetivos que se reformulan

La clasificación multi-output queda descartada como enfoque principal en esta fase. La razón es que las etiquetas no nacen como clases, sino como scores continuos derivados de patrones observados en la timeline. Al discretizar esos scores, se introducen umbrales artificiales y aparece una zona intermedia difícil de interpretar. En los experimentos previos, esta zona se trató mediante una clase ambigüa, pero acabó dificultando tanto el aprendizaje como la evaluación.

Además, la clasificación hace que errores muy distintos puedan penalizarse de la misma forma. Por ejemplo, si una partida tiene un score real de 0.62, una predicción de 0.58 debería considerarse mucho mejor que una predicción de 0.15. En una formulación puramente clasificatoria, esa diferencia puede perderse si ambas predicciones caen al otro lado de un umbral.

La regresión continua permite conservar la naturaleza gradual de la etiqueta y medir el error según la distancia entre predicción y valor real. Por este motivo, la fase actual del proyecto se centra en predecir un score continuo de roaming del support en el rango [0, 1].

3.3 Objetivo final del prototipo

El objetivo aplicado sigue siendo construir un prototipo por terminal. La idea final es que el usuario introduzca manualmente un draft, incluyendo campeones, roles y contexto relevante, y que el sistema devuelva scores estimados junto con frases interpretables.

Por ejemplo, en la tarea de support, el sistema no debería devolver únicamente un número como "0.37", sino traducirlo a una lectura comprensible: "perfil de roaming moderado; el support puede moverse, pero el draft no sugiere una tendencia extrema a abandonar la botlane".

De este modo, el prototipo final no se plantea solo como un modelo predictivo, sino como una herramienta de interpretación previa a la partida.

4 TRABAJO PREVIO REALIZADO

4.1 Pipeline inicial

En la primera etapa, tras plantear la propuesta inicial, se desarrolló un pipeline completo que incluía:

- 1) Recolección de partidas y timelines desde la API de

Riot Games.

- 2) Transformación de los JSON de información originales en características de draft y etiquetas.
- 3) Cálculo de scores de jungler, support y equipo.
- 4) Discretización de scores y entrenamiento de un modelo de clasificación.
- 5) Análisis exploratorio, gráficas y comparación entre distintas ventanas de minutos.

La mayoría de este trabajo no se descarta. Al contrario, sirve como base para justificar el cambio de enfoque. El análisis exploratorio permitió detectar problemas en la definición de etiquetas, estudiar la estabilidad de distintas ventanas temporales y observar fenómenos relevantes del mapa, como diferencias entre los equipos azul y rojo en el acceso a objetivos neutrales y en algunos scores observados.

4.2 Estabilidad de ventanas temporales

La primera fase experimental estudió si la ventana temporal usada para construir las etiquetas afectaba a su estabilidad y al rendimiento del modelo. Para ello, se probaron distintas ventanas de juego temprano: desde el minuto 0 hasta los minutos 6, 8, 10, 12 y 14.

La conclusión principal fue que las tareas no se estabilizan al mismo ritmo:

- 1) La etiqueta de jungler tendía a estabilizarse antes.
- 2) La etiqueta de support necesitaba una ventana algo más amplia, alrededor de 12 minutos.
- 3) La tendencia de equipo era la más inestable.

Este resultado fue importante porque mostró que no era adecuado tratar las tres tareas como equivalentes. Cada una describe un fenómeno diferente y, por tanto, puede requerir una definición temporal distinta.

También se observó que, en la mayoría de los casos, la inestabilidad no procedía de cambios directos entre clases extremas, sino de transiciones hacia o desde la zona central ambigua. Es decir, el problema principal no era que una partida pasara de un patrón claramente bajo a otro claramente alto, sino que muchos ejemplos quedaban cerca de la frontera entre una clase definida y una clase intermedia.

4.3 Problema de la clase ambigua

La segunda fase analizó el impacto de la clase intermedia. Para ello, se compararon tres esquemas de etiquetado:

- 1) Esquema ternario: conservaba las dos clases extremas, definidas a partir de los cuartiles 20 y 80, y añadía una clase intermedia.
- 2) Esquema binario limpio: descartaba los ejemplos situados en la clase intermedia y conservaba únicamente los extremos.
- 3) Esquema binario completo: no descartaba los ejemplos intermedios, pero los asignaba a una de las dos clases extremas.

Los experimentos mostraron que eliminar o reducir la amplitud de la clase intermedia mejoraba las métricas de clasificación, especialmente cuando solo se conservaban ejemplos extremos. Esto confirmó que la clase central representaba una zona de ruido, indefinición o baja separabilidad para el modelo.

Sin embargo, esta conclusión reveló también una limitación importante. Si la mejor forma de obtener buenos resultados de clasificación era eliminar gran parte de la zona central, entonces la formulación discreta estaba desaprovechando información de muchas partidas. En lugar de resolver el problema, la clasificación lo simplificaba artificialmente.

Por este motivo, el siguiente paso fue abandonar la predicción de clases y predecir directamente el score continuo. De esta manera, la zona central deja de ser una clase problemática y pasa a formar parte natural de una escala gradual.

5 METODOLOGÍA ACTUAL

5.1 Datos y unidad de análisis

La API de Riot Games proporciona endpoints para obtener información de partidas y timelines. Además, se utiliza Data Dragon para datos estáticos y recursos asociados al juego, como campeones, hechizos, runas e iconos [6].

La unidad de análisis es (match_id, team_id). Esto significa que cada partida genera dos observaciones: una desde la perspectiva del equipo azul y otra desde la perspectiva del equipo rojo. Esta formulación permite estudiar patrones de equipo sin reducir la partida a una única observación global.

Como se ha explicado, se separa estrictamente información pre-partida y comportamiento observado para evitar data leakage. Las variables del draft se usan como entrada del modelo, mientras que las variables de la timeline se emplean únicamente para construir la etiqueta observada.

5.2 Nueva etiqueta de support

La etiqueta actual mide el perfil de roaming del support entre el minuto 5 y el minuto 12. El score se define en el rango [0, 1], donde valores bajos indican un support más anclado a botlane y al botlaner, mientras que valores altos indican una mayor tendencia observada a abandonar la línea o jugar lejos del botlaner.

La heurística combina tres componentes, con los siguientes pesos en la versión actual:

- 1) Proporción de instantes de la timeline fuera de botlane y de sus alrededores próximos: 0.45.
- 2) Proporción de instantes de la timeline lejos del botlaner: 0.35.
- 3) Componente de diferencia relativa de experiencia entre support y botlaner: 0.20.

El tercer componente no compara recursos entre equipos, sino la relación interna entre el support y su compañero de línea. Su objetivo es capturar situaciones en las que el support se separa del botlaner durante la fase temprana, lo que puede reflejar presencia en otras zonas del mapa. Aun así, este componente es uno de los candidatos a revisión durante el tuning de la etiqueta, precisamente porque puede estar más influido por el desarrollo concreto de la partida que las variables puramente espaciales.

La formulación de esta etiqueta constituye una parte central del trabajo propio: no se dispone de un ground truth oficial para el roaming del support, por lo que la

etiqueta debe diseñarse, analizarse y revisarse experimentalmente.

5.3 Comparación experta por campeón

Para comprobar si la etiqueta observada tiene sentido cualitativo, se construyó una referencia experta manual para 47 campeones con presencia habitual o plausible en rol de support. Esta tabla incluye:

- 1) Nombre del campeón.
- 2) Arquetipo experto.
- 3) Score experto o esperado de roaming en [0, 1].
- 4) Confianza de la etiqueta.
- 5) Notas justificativas.

Un ejemplo simplificado de cinco filas, sin incluir las notas, es:

Campeón	Arquetipo experto	Score	Conf.
Bard	Roaming specialist	0.95	0.95
Alistar	Engage roamer	0.82	0.90
Braum	Protective warden	0.48	0.75
Sona	Scaling enchanter	0.22	0.8
Yuumi	Adc enabler	0.08	0.95

La comparación observada vs experta no debe presentarse como validación contra un ground truth oficial, sino como validación cualitativa inicial. El objetivo es comprobar si campeones que por conocimiento del dominio deberían aparecer como más orientados al roaming, como Bard, Pyke, Nautilus o Rakan, quedan por encima de campeones más anclados al botlaner, como Yuumi, Soraka, Sona o Milio.

Esta referencia experta no procede de internet ni de una fuente oficial externa. Es una primera curación manual realizada para el proyecto a partir de conocimiento del dominio: identidad del campeón, rol habitual, kit de habilidades y patrón esperado de juego. Por tanto, la confianza tiende a ser mayor en los extremos evidentes y menor en campeones intermedios, situacionales o dependientes del parche.

5.4 Modelo MLP

La primera baseline utiliza una MLP con codificación OneHot. Esta decisión se tomó como punto de partida porque la mayor parte de variables de entrada son categóricas. OneHotEncoder es un transformador que convierte variables categóricas en columnas binarias, una representación común en aprendizaje automático [7].

El modelo usa MSE como función de pérdida, que mide el error cuadrático medio entre la predicción y el valor objetivo [9]. Esta función es adecuada para una primera formulación de regresión porque penaliza más los errores grandes que los pequeños, la cual cosa tiene sentido con la motivación principal del cambio desde clasificación.

El split de entrenamiento y validación se hace por match_id, no por filas aleatorias independientes. Esto evita que una misma partida pueda aparecer parcialmente en entrenamiento y parcialmente en validación, ya que cada partida genera una observación por equipo. La librería scikit-learn proporciona GroupShuffleSplit precisamente para separar conjuntos de datos según grupos [10].

6 RESULTADOS OBTENIDOS

6.1 Snapshot del dataset

Para construir el dataset se han aplicado varios criterios de filtrado. Entre ellos, las partidas deben durar al menos 15 minutos y deben contener información válida de roles, especialmente para support y jungler.

El primer archivo clave del cambio metodológico es el frame-state. Se trata de una tabla derivada de las timelines de Riot que resume el estado de la partida en distintos instantes: minuto, equipo, campeón, rol, posición, experiencia y variables necesarias para medir si el support permanece en botlane, se aleja del botlaner o se desplaza hacia otras zonas del mapa. Es caro de construir porque requiere leer las partidas y sus timelines originales, pero permite después recalcular etiquetas con distintas heurísticas sin volver a procesar todos los JSON. El frame-state, por tanto, no es la entrada del modelo, sino una representación transformada del comportamiento observado que se utiliza para calcular la etiqueta que actúa como valor objetivo.

El snapshot completo actual contiene:

Métrica	Valor
Partidas detectadas	171386
Partidas válidas en frame-state	168564
Filas de frame-state	9823624
Partidas descartadas por duración < 15 minutos	2805
Partidas descartadas por datos corruptos	17

El segundo archivo clave es draft_features. A diferencia del frame-state, estas variables sí representan información disponible antes de empezar la partida: campeones seleccionados por cada equipo, roles, lado del mapa y otros campos derivados del draft. Son la base tabular que se combinará con la etiqueta de support para construir el input final de entrenamiento.

El draft_features completo contiene:

Métrica	Valor
Partidas procesadas	171386
Partidas válidas	168565
Filas de draft	337130
Observaciones únicas (match_id, team_id)	337130

Esto confirma que se genera una observación por equipo. La diferencia de una partida entre draft_features y frame-state se debe a una timeline inválida y no afecta de forma relevante al análisis completo.

6.2 Salud de la etiqueta

La etiqueta continua de support baseline, calculada entre los minutos 5 y 12, se generó para 337094 observaciones. No hay nulos y todos los valores se mantienen dentro del rango [0, 1].

Métrica	Valor
Filas etiquetadas	337094
Media	0.2828

Desviación típica	0.1722
Mínimo	0.0000
Q05	0.0385
Q25	0.1500
Mediana	0.2636
Q75	0.3917
Q95	0.5929
Q99	0.7490
Máximo	1.0000
Porcentaje en 0	1.97%
Porcentaje en 1	0.037%

Como se observa, la distribución está concentrada hacia la izquierda, pero no colapsada. Esto no es necesariamente negativo: el score mide intensidad de roaming real en una partida concreta, y la mayoría de supports no juegan roaming extremo en todos los frames entre los minutos 5 y 12, al menos no con la definición actual de la etiqueta.

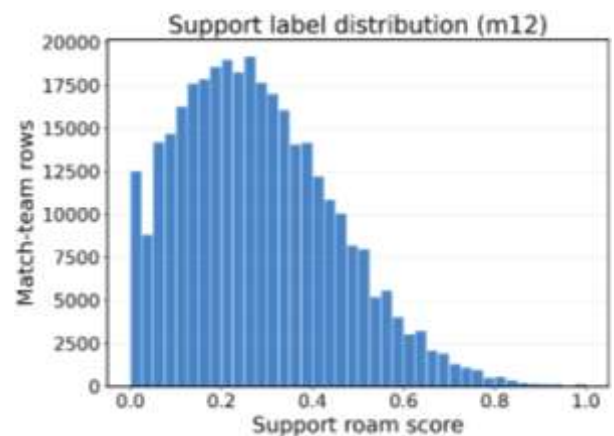


Fig. 2. Distribución del score de la etiqueta de support. El eje horizontal representa el score en [0, 1] y el eje vertical el número de observaciones.

También aparece una ligera asimetría por lado:

Lado	Media	Mediana	Desviación
Azul	0.2881	0.2700	0.1731
Rojo	0.2775	0.2577	0.1711

Esta diferencia es coherente con observaciones tempranas del proyecto. Los supports empiezan normalmente ligados a botlane junto al botlaner. El dragón está situado en la zona inferior del río, mientras que las larvas del vacío aparecen en la parte superior. Si el equipo azul reclama las larvas primero en torno al 59.38% de las partidas, es normal pensar que sus supports realicen ligeramente más movimientos hacia zonas alejadas de botlane durante la fase temprana del juego.

Por tanto, dado que los accesos a las zonas de dragón y larvas no son perfectamente simétricos, parte de la asimetría observada puede reflejar la estructura real del mapa y no necesariamente un error de la etiqueta.

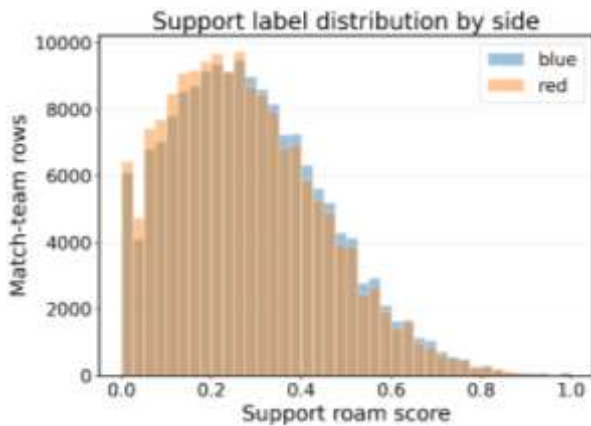


Fig. 3. Distribución del score de la etiqueta de support por lado del mapa. Se observa un ligero desplazamiento de la distribución según el lado.

6.3 Comparación observada frente a experta

Para comparar resultados por campeón, se agregó la etiqueta generada calculando el score medio observado de cada campeón en rol de support. Posteriormente, estos valores se compararon con la referencia experta manual descrita en la metodología.

La comparación se realizó sobre 47 campeones y obtuvo una correlación de Pearson de 0.795 y una correlación de Spearman de 0.825. Estos valores indican una correlación alta entre el ranking experto y el ranking observado. Sin embargo, la escala observada está mucho más comprimida: mientras que la referencia experta utiliza todo el rango $[0, 1]$, las medias observadas por campeón no llegan a 0.45.

En el scatter, la línea discontinua es una referencia visual escalada al rango observado. El eje experto llega a 1, pero la media observada por campeón no llega a 0.45; por eso la línea apunta al límite superior útil del eje Y y no al valor 1. El color introduce una tercera variable: la confianza asignada durante el etiquetaje experto. Los puntos más verdes corresponden a campeones cuya identidad de support o roaming se considera más clara. En cambio, los puntos más rojos o amarillos indican casos más discutibles o situacionales. Esto permite comprobar visualmente si, por ejemplo, las mayores desviaciones se concentran en etiquetas de menor confianza.

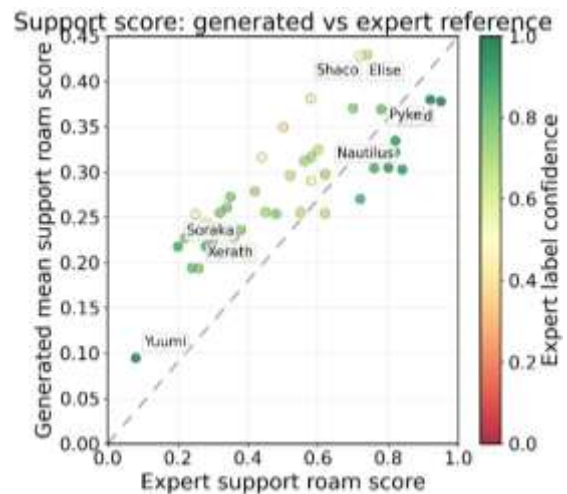


Fig. 4. Scatterplot del score medio observado frente al score experto, por campeón. Se observa una correlación positiva fuerte, aunque con una escala observada más comprimida.

6.4 Primera MLP

La primera MLP se entrenó sobre el dataset completo con la siguiente configuración:

Elemento	Valor
Filas train	269674
Filas validación	67420
Dimensión One-Hot	1796
Capas ocultas	256 -> 128
Dropout	0.2
Loss	MSE
Mejor época	6 ^a

Las métricas en validación fueron las siguientes:

Métrica	Valor
MSE	0.02543
RMSE	0.15947
MAE	0.12721
R2	0.13068
Pearson	0.3633
Spearman	0.3568

Frente a predecir siempre la media de validación ($MSE=0.02925$, $MAE=0.13791$), la MLP mejora aproximadamente un 13% en MSE y un 8% en MAE. Esto confirma que el draft contiene señal predictiva, aunque la señal aprendida todavía es limitada.

La curva de entrenamiento muestra sobreajuste temprano. La pérdida en entrenamiento sigue bajando después de la época 6, pero la pérdida de validación empieza a subir. Esto indica que el modelo aprende patrones específicos del conjunto de entrenamiento que no generalizan igual de bien.

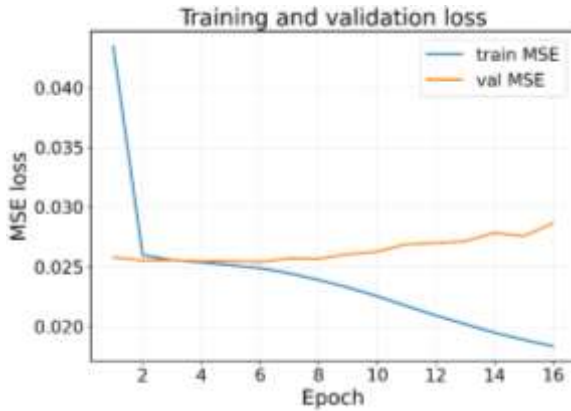


Fig. 5. Curvas de entrenamiento y validación del modelo. Se observa sobreajuste temprano a partir de la mejor época de validación.

El scatter true-vs-pred confirma una tendencia clara a la regresión a la media. Las predicciones se concentran en una banda central: sobreestiman scores bajos y subestiman scores altos. Por tanto, la relación positiva entre valor real y predicción existe, pero aparece debilitada por la compresión de las predicciones.

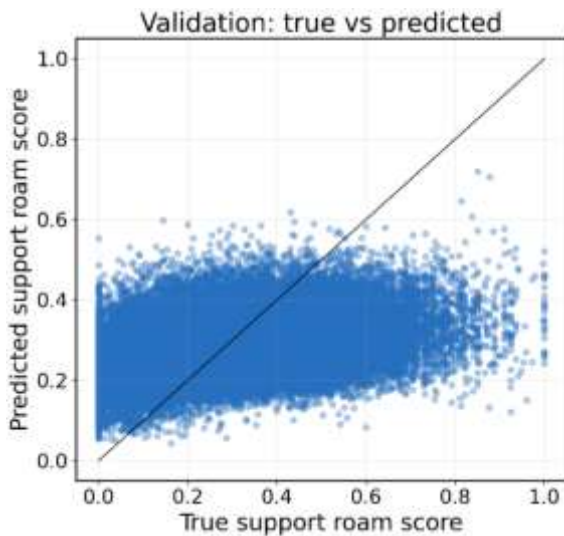


Fig. 6. Scatterplot del score de support predicho frente al score real. La nube muestra compresión de las predicciones hacia valores medios.

Finalmente, el error por rango de score muestra que la MLP se comporta mejor en la zona media y falla especialmente en los extremos. El hecho de que el error sea menor en la zona media no implica necesariamente que haya que forzar una distribución plana del valor objetivo. En primer lugar, hay más observaciones en la zona media, por lo que el modelo recibe más ejemplos de esos rangos. En segundo lugar, la MLP tiende a predecir valores cercanos a la media, lo que reduce el error en scores intermedios, pero lo aumenta en los extremos.

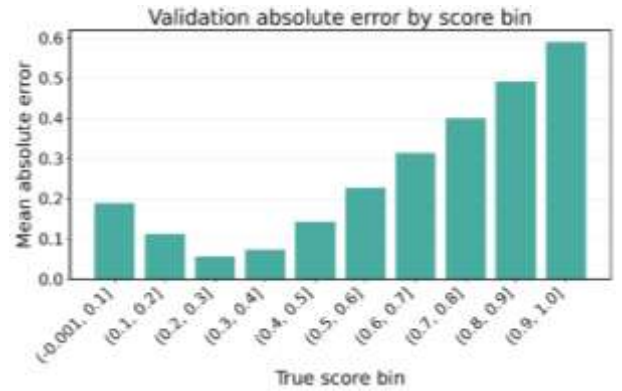


Fig. 7. Error medio absoluto por rango de score real. El error crece especialmente en los extremos de la distribución.

7 DIFICULTADES ENCONTRADAS

7.1 Intención de draft vs. patrones observables

El proyecto intenta predecir comportamiento temprano a partir del draft. Sin embargo, la etiqueta se calcula a partir de comportamiento observado durante la partida. Esto introduce una diferencia conceptual importante: el draft representa una predisposición estratégica, mientras que la timeline refleja una ejecución concreta, afectada por decisiones de los jugadores, errores, coordinación, enfrentamientos concretos y ventajas acumuladas.

Para reducir esta diferencia se han tomado varias decisiones de diseño. En primer lugar, el dataset procede de jugadores de muy alto nivel, por lo que se asume que la ejecución observada se aproxima mejor a la intención estratégica del draft que en partidas de menor nivel. En jugadores menos expertos, es más probable que la composición no se utilice de acuerdo con sus fortalezas y debilidades esperadas.

En segundo lugar, la etiqueta de support prioriza métricas espaciales, como salir de botlane, alejarse del botlaner o moverse por zonas relevantes del mapa, en lugar de usar directamente indicadores de ventaja entre equipos, como oro global, kills u objetivos acumulados. Esta decisión intenta evitar que el valor objetivo mida simplemente si un equipo va ganando o perdiendo.

En tercer lugar, se analiza una ventana temprana situada entre los minutos 5 y 12. Esta ventana busca capturar patrones iniciales antes de que la partida quede demasiado condicionada por ventajas acumuladas. Aun así, la limitación no desaparece: la etiqueta sigue midiendo ejecución real, no intención pura.

7.2 Definición de etiquetas continuas

Una de las principales dificultades del proyecto ha sido definir etiquetas que sean interpretables, calculables de forma automática y útiles para entrenar modelos. Las primeras versiones discretizaban scores continuos en clases, pero esto introducía umbrales arbitrarios y una clase intermedia ambigua difícil de aprender.

El cambio a regresión continua reduce este problema, pero no lo elimina por completo. La etiqueta de support sigue dependiendo de decisiones de diseño: qué se

considera “estar fuera de botlane”, qué distancia respecto al botlaner indica separación relevante, qué ventana temporal se utiliza y qué peso recibe cada componente de la heurística.

Por este motivo, la definición de la etiqueta no se considera terminada. El siguiente paso será realizar experimentos sobre sus componentes para comprobar qué formulaciones producen etiquetas más coherentes, estables y aprendibles.

7.3 Predicciones comprimidas de la MLP

La primera MLP entrenada con el nuevo enfoque aprende señal, pero sus predicciones tienen mucha menos varianza que el valor objetivo. En validación, el valor objetivo real tiene una desviación típica de 0.1710, mientras que las predicciones de la MLP tienen una desviación típica inferior a 0.07.

Esto indica que el modelo tiende a la media: evita hacer predicciones extremas y se concentra en valores intermedios. Este comportamiento es habitual en modelos de regresión cuando el input disponible explica parte de la variabilidad, pero no toda. En este caso, el draft contiene señal predictiva, pero no determina por completo cómo se ejecutará la partida.

La siguiente fase debe comprobar si esta compresión mejora con tuning, regularización, embeddings, feature enrichment o, si se acuerda con el tutor, una representación secuencial basada en la timeline.

8. PRÓXIMOS PASOS TÉCNICOS

El siguiente paso técnico es ejecutar un tuning OAT (One At-a-Time). La idea es probar, de forma controlada y modificando una variable cada vez, tanto hiperparámetros de la red como variantes de la etiqueta de support. En la parte del modelo se revisarán aspectos como regularización, capacidad de la red, dropout, learning rate o batch size. En la parte de la etiqueta se probarán cambios en los pesos de la heurística, la ventana temporal, el minuto de inicio y los umbrales espaciales.

Si la MLP mejora de forma consistente, el siguiente bloque de trabajo será aplicar feature enrichment o utilizar embeddings. La motivación es que One-Hot representa cada campeón como una categoría independiente, sin codificar relaciones entre campeones. En cambio, una representación enriquecida podría ayudar al modelo a capturar que campeones con arquetipos similares, como supports de engage o enchanter defensivos, pueden tener patrones de comportamiento parecidos.

Después de estabilizar la etiqueta de support y aprovechar el conocimiento obtenido durante esta fase, se reintroducirán nuevas versiones continuas de las etiquetas de jungla y equipo. La idea es evitar repetir los problemas detectados en las primeras fases, especialmente la discretización prematura y la clase ambigua.

Finalmente, si el tiempo disponible lo permite y se acuerda con el tutor, se explorarán arquitecturas secuenciales como RNN o incluso LSTM. Esto se debe a que la timeline de Riot Games es una secuencia de snapshots del

estado de la partida a cada minuto. La pregunta experimental sería si modelar directamente la evolución temporal de posiciones, recursos y eventos permite explicar mejor las etiquetas que una representación basada únicamente en la información del draft.

9. PLANNING HASTA EL FINAL DE PROYECTO

Período	28/04 – 03/05
Objetivo	Tuning OAT de MLP + etiqueta support
Tarea	Probar regularización, capacidad, dropout, LR, batch size, pesos, ventana y umbral de distancia al botlaner
Entregable esperado	Tabla comparativa por val_mse, curvas, mejores heurísticas y etiqueta candidata
Criterio de éxito	Mejorar o justificar configuración base y seleccionar etiqueta support candidata

Período	04/05 – 10/05
Objetivo	Embeddings y feature enrichment inicial
Tarea	Diseñar features semánticas de campeones, runas, hechizos, arquetipos y tags
Entregable esperado	Comparación One-Hot vs enriched/embeddings
Criterio de éxito	Mejora medible o descarte con justificación

Período	11/05 – 17/05
Objetivo	Refinar representación y cerrar soporte
Tarea	Analizar importancia de features, ajustar representación enriched/embedding y consolidar etiqueta support candidata
Entregable esperado	Feature set candidato + decisión sobre representación de entrada
Criterio de éxito	Representación más informativa o descarte razonado frente al One-Hot

Período	18/05 – 24/05
Objetivo	Informe de Progreso II
Tarea	Integrar tuning, embeddings, etiqueta support candidata, decisiones y figuras finales parciales
Entregable esperado	Informe II revisado
Criterio de éxito	Documento listo para entrega del 24/05

Período	25/05 – 31/05
Objetivo	Redefinir etiqueta de jungla
Tarea	Diseñar versión continua de jungla, ejecutar pruebas de generación y revisar distribución
Entregable esperado	Propuesta de etiqueta jungle + gráficas de salud
Criterio de éxito	Etiqueta coherente, no degenerada y compatible con pipeline

Período	01/06 - 07/06
Objetivo	Redefinir etiqueta de equipo
Tarea	Diseñar versión continua de equipo, ejecutar pruebas de generación y revisar distribución
Entregable esperado	Propuesta de etiqueta equipo + gráficas de salud
Criterio de éxito	Etiqueta coherente, no degenerada y compatible con pipeline

Período	08/06 - 14/06
Objetivo	Integración multi-output y decisión RNN con el tutor
Tarea	Integrar support/jungla/equipo. Después, reunirse con tutor y decidir si explorar modelo secuencial
Entregable esperado	Modelo candidato, decisión documentada y estructura de memoria
Criterio de éxito	Siguiente paso aprobado o descartado por alcance

Período	15/06 - 21/06
Objetivo	CLI de entrada manual y traducción del score a texto
Tarea	Implementar CLI para introducir draft manualmente, cargar modelo candidato y traducir scores a frases interpretables.
Entregable esperado	CLI mínimo con frases interpretables
Criterio de éxito	Usuario introduce el draft y recibe una lectura clara

Período	22/06 - 28/06
Objetivo	Cierre final
Tarea	Conclusiones, limitaciones, memoria, presentación
Entregable esperado	Dossier final + informe + presentación
Criterio de éxito	Entrega final revisada

- [4] G. Wallner, L. Wang, and C. Dormann, "Visualizing the Spatio-Temporal Evolution of Gameplay using Storyline Visualization: A Study with League of Legends," *Proc. ACM Hum.-Comput. Interact.*, vol. 7, CHI PLAY, pp. 1002-1024, 2023.
- [5] Y. Chen, J. Wu, Y. Wu, and D. Liu, "T-Foresight: Interpret moving strategies based on context-aware trajectory prediction," *Visual Informatics*, vol. 9, no. 3, art. 100261, 2025.
- [6] Riot Games, "League of Legends Developer Documentation: Data Dragon," Riot Developer Portal. [Online]. Available: <https://developer.riotgames.com/docs/lol>. Accessed: Apr. 25, 2026.
- [7] scikit-learn developers, "OneHotEncoder," scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>. Accessed: Apr. 25, 2026.
- [8] PyTorch contributors, "MSELoss," PyTorch documentation. [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>. Accessed: Apr. 25, 2026.
- [9] scikit-learn developers, "GroupShuffleSplit," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html. Accessed: Apr. 25, 2026.

REFERENCIAS

- [1] J.-A. Hitar-García, L. Moran-Fernandez, and V. Bolon-Canedo, "Machine Learning Methods for Predicting League of Legends Game Outcome," *IEEE Transactions on Games*, vol. 15, no. 2, pp. 171-181, 2023.
- [2] H. Lee, D. Hwang, H. Kim, B. Lee, and J. Choo, "DraftRec: Personalized Draft Recommendation for Winning in Multi-Player Online Battle Arena Games," in *Proc. ACM Web Conf. 2022 (WWW '22)*, pp. 3428-3439, 2022.
- [3] A. M. Rama, V. Rodríguez-Fernandez, and D. Camacho, "Finding Behavioural Patterns Among League of Legends Players Through Hidden Markov Models," in *Applications of Evolutionary Computation*, *Lecture Notes in Computer Science*, vol. 12104, pp. 419-430, 2020.